

JAVA 2ND ROUND QUESTIONS AND ANSWERS

54. Given two dimensional matrix of integer and print the rectangle can be formed using given indices and also find the sum of the elements in the rectangle

Input: `mat[M][N] = {{1, 2, 3, 4, 6}, {5, 3, 8, 1, 2}, {4, 6, 7, 5, 5}, {2, 4, 8, 9, 4} }`;

index = (2, 0) and (3, 4)

Output:

Rectangle

4 6 7 5 5

2 4 8 9 4

sum 54.

ANSWER:

```
public class RectangleSum {
    public static void main(String[] args) {
        int[][] mat = {
            {1, 2, 3, 4, 6},
            {5, 3, 8, 1, 2},
            {4, 6, 7, 5, 5},
            {2, 4, 8, 9, 4}
        };

        int[] index1 = {2, 0};
        int[] index2 = {3, 4};

        printRectangleAndSum(mat, index1, index2);
    }

    public static void printRectangleAndSum(int[][] mat, int[] index1, int[] index2) {
        int sum = 0;
        int rowStart = Math.min(index1[0], index2[0]);
        int rowEnd = Math.max(index1[0], index2[0]);
        int colStart = Math.min(index1[1], index2[1]);
        int colEnd = Math.max(index1[1], index2[1]);

        System.out.println("Rectangle:");
        for (int i = rowStart; i <= rowEnd; i++) {
            for (int j = colStart; j <= colEnd; j++) {
                System.out.print(mat[i][j] + " ");
                sum += mat[i][j];
            }
            System.out.println();
        }
        System.out.println("Sum: " + sum);
    }
};
```

55. Find the result subtraction, multiplication, division of two integers using + operator.

Input: 6 and 4

output:

addition $6+4 = 10$, subtraction $6+(-4) = 2$, multiplication = 24, division = 1

Input : -8 and -4

Output:

addition $-8+(-4) = -12$, subtraction $(-8)+(-(-4)) = -4$, multiplication = 32, division = 2.

ANSWER

```
public class ArithmeticOperations {
    public static void main(String[] args) {
        int num1 = 6;
        int num2 = 4;

        System.out.println("Input: " + num1 + " and " + num2);
        System.out.println("Output:");
        System.out.println("addition " + num1 + "+" + num2 + " = " + add(num1, num2));
        System.out.println("subtraction " + num1 + "+" + negate(num2) + " = " + subtract(num1, num2));
        System.out.println("multiplication = " + multiply(num1, num2));
        System.out.println("division = " + divide(num1, num2));
    }

    // Addition using only '+'
    public static int add(int a, int b) {
        return a + b;
    }

    // Subtraction using only '-'
    public static int subtract(int a, int b) {
        return a + negate(b);
    }

    // Multiplication using only '*'
    public static int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < Math.abs(b); i++) {
            result += a;
        }
        if ((a < 0 && b > 0) || (a > 0 && b < 0)) {
            result = negate(result);
        }
        return result;
    }

    // Division using only '/'
    public static int divide(int a, int b) {
        int result = 0;
        int dividend = Math.abs(a);
        int divisor = Math.abs(b);
        while (dividend >= divisor) {
            dividend = subtract(dividend, divisor);
            result = add(result, 1);
        }
        if ((a < 0 && b > 0) || (a > 0 && b < 0)) {
            result = negate(result);
        }
        return result;
    }

    // Negate a number
    public static int negate(int a) {
        return add(~a, 1);
    }
}
```

56..Given a sentence of string, in that remove the palindrome words and print the remaining.

Input:

He did a good deed

Output:

He good

Input:

Hari speaks malayalam

Output:

Hari speaks

ANSWER:

```
public class RemovePalindromes {
    public static void main(String[] args) {
        String sentence1 = "He did a good deed";
        String sentence2 = "Hari speaks malayalam";

        System.out.println(removePalindromes(sentence1));
        System.out.println(removePalindromes(sentence2));
    }

    public static String removePalindromes(String sentence) {
        StringBuilder result = new StringBuilder();
        String[] words = sentence.split(" ");

        for (String word : words) {
            if (!isPalindrome(word)) {
                result.append(word).append(" ");
            }
        }

        // Remove trailing space
        if (result.length() > 0) {
            result.deleteCharAt(result.length() - 1);
        }

        return result.toString();
    }

    public static boolean isPalindrome(String word) {
        int left = 0;
        int right = word.length() - 1;

        while (left < right) {
            if (word.charAt(left) != word.charAt(right)) {
                return false;
            }
        }
    }
}
```

```

        left++;
        right--;
    }

    return true;
}
}

```

57..Given two dates, find total number of days between them.

Input: dt1 = {10, 2, 2014} dt2 = {10, 3, 2015}

Output: 393

dt1 represents "10-Feb-2014" and dt2 represents "10-Mar-2015" The difference is 365 + 28

Input: dt1 = {10, 2, 2000} dt2 = {10, 3, 2000}

Output: 29

Note that 2000 is a leap year

Input: dt1 = {10, 2, 2000} dt2 = {10, 2, 2000}

Output: 0

Both dates are same

Input: dt1 = {1, 2, 2000}; dt2 = {1, 2, 2004};

Output: 1461

Number of days is $365 \times 4 + 1$

ANSWER;

```

public class DaysBetweenDates {
    public static void main(String[] args) {
        int[] dt1 = {10, 2, 2014};
        int[] dt2 = {10, 3, 2015};
        System.out.println(daysBetween(dt1, dt2)); // Output: 393

        int[] dt3 = {10, 2, 2000};
        int[] dt4 = {10, 3, 2000};
        System.out.println(daysBetween(dt3, dt4)); // Output: 29

        int[] dt5 = {10, 2, 2000};
        int[] dt6 = {10, 2, 2000};
        System.out.println(daysBetween(dt5, dt6)); // Output: 0

        int[] dt7 = {1, 2, 2000};
        int[] dt8 = {1, 2, 2004};
        System.out.println(daysBetween(dt7, dt8)); // Output: 1461
    }

    public static int daysBetween(int[] dt1, int[] dt2) {
        // Days in each month
        int[] monthDays = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        int days1 = dt1[0];
        int month1 = dt1[1];
        int year1 = dt1[2];
    }
}

```

```

int days2 = dt2[0];
int month2 = dt2[1];
int year2 = dt2[2];

// Count total number of days from the start of the years
int totalDays1 = days1;
int totalDays2 = days2;

// Add days from months of dt1
for (int i = 0; i < month1 - 1; i++) {
    totalDays1 += monthDays[i];
}

// Add days from months of dt2
for (int i = 0; i < month2 - 1; i++) {
    totalDays2 += monthDays[i];
}

// Adjust for leap years
totalDays1 += countLeapYears(year1, month1);
totalDays2 += countLeapYears(year2, month2);

// Add remaining days of the given month
totalDays1 += year1 * 365;
totalDays2 += year2 * 365;

// If dt1 comes after dt2, swap them
if (totalDays1 > totalDays2) {
    int[] temp = dt1;
    dt1 = dt2;
    dt2 = temp;

    int tempDays = totalDays1;
    totalDays1 = totalDays2;
    totalDays2 = tempDays;
}

// Add extra day for leap year if dt1 is a leap year
if (isLeap(year1, month1) && month1 <= 2) {
    totalDays1++;
}

// Add total days for the years in between dt1 and dt2
for (int i = year1 + 1; i < year2; i++) {
    totalDays1 += 365;
    if (isLeap(i, 0)) {
        totalDays1++;
    }
}

```

```

        // Add remaining days of the given month for dt2
        totalDays2 += days2;

        return totalDays2 - totalDays1;
    }

    // Function to check if a year is leap year
    public static boolean isLeap(int year, int month) {
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
            return true;
        }
        return false;
    }

    // Function to count leap years
    public static int countLeapYears(int year, int month) {
        if (month <= 2) {
            year--;
        }
        return year / 4 - year / 100 + year / 400;
    }
}

```

58. Let 1 represent 'A', 2 represents 'B', etc. Given a digit sequence, count the number of possible decodings of the given digit sequence.

Examples:

Input: digits[] = "121"

Output: 3 // The possible decodings are "ABA", "AU", "LA"

Input: digits[] = "1234" **Output: 3**

// The possible decodings are "ABCD", "LCD", "AWD"

ANSWER;

```

public class DecodeWays {
    public static void main(String[] args) {
        String digits1 = "121";
        System.out.println(countDecodings(digits1)); // Output: 3

        String digits2 = "1234";
        System.out.println(countDecodings(digits2)); // Output: 3
    }

    public static int countDecodings(String digits) {
        int n = digits.length();
        if (n == 0 || (n == 1 && digits.charAt(0) == '0')) {
            return 0;
        }
        if (n == 1) {

```

```

        return 1;
    }

    // Initialize an array to store the number of decodings for each digit
    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = digits.charAt(0) != '0' ? 1 : 0;

    for (int i = 2; i <= n; i++) {
        // Check if the single digit forms a valid encoding
        if (digits.charAt(i - 1) != '0') {
            dp[i] += dp[i - 1];
        }
        // Check if the two digits form a valid encoding
        int twoDigits = Integer.parseInt(digits.substring(i - 2, i));
        if (twoDigits >= 10 && twoDigits <= 26) {
            dp[i] += dp[i - 2];
        }
    }

    return dp[n];
}
}

```

59.. Print all possible words from phone digits
ANSWER;

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class PhoneDigitsWords {
    public static void main(String[] args) {
        String digits = "23";
        System.out.println("Possible words from phone digits:");
        List<String> words = letterCombinations(digits);
        for (String word : words) {
            System.out.println(word);
        }
    }

    public static List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();
        if (digits == null || digits.length() == 0) {
            return result;
        }
    }
}

```

```

// Mapping of digits to letters
Map<Character, String> phoneMap = new HashMap<>();
phoneMap.put('2', "abc");
phoneMap.put('3', "def");
phoneMap.put('4', "ghi");
phoneMap.put('5', "jkl");
phoneMap.put('6', "mno");
phoneMap.put('7', "pqrs");
phoneMap.put('8', "tuv");
phoneMap.put('9', "wxyz");

backtrack(result, phoneMap, digits, "", 0);
return result;
}

private static void backtrack(List<String> result, Map<Character, String> phoneMap, String
digits, String combination, int index) {
    if (index == digits.length()) {
        result.add(combination);
        return;
    }

    char digit = digits.charAt(index);
    String letters = phoneMap.get(digit);
    for (char letter : letters.toCharArray()) {
        backtrack(result, phoneMap, digits, combination + letter, index + 1);
    }
}
}

```

60. Print longest sequence between same character

Ex I/p abccccbba

O/p 8 (from a to a)

I/p aaaaaaaa

O/p 6

ANSWER:

```

public class LongestSequenceBetweenSameCharacter {
    public static void main(String[] args) {
        String input1 = "abccccbba";
        String input2 = "aaaaaaa";

        System.out.println("Longest sequence between same character in \"" + input1 + "\": " +
longestSequence(input1)); // Output: 8
        System.out.println("Longest sequence between same character in \"" + input2 + "\": " +
longestSequence(input2)); // Output: 6
    }

    public static int longestSequence(String input) {
        if (input == null || input.length() == 0) {

```



```

        return 0;
    }

    int maxCount = 1;
    int count = 1;

    for (int i = 1; i < input.length(); i++) {
        if (input.charAt(i) == input.charAt(i - 1)) {
            count++;
        } else {
            maxCount = Math.max(maxCount, count);
            count = 1;
        }
    }

    // Check if the last sequence is the longest
    maxCount = Math.max(maxCount, count);

    return maxCount;
}
}

```

61..sort the array odd numbers in ascending and even numbers in descending.

I/p 5 8 11 6 2 1 7

O/p 1 5 7 11 8 6 2

ANSWER:

```

import java.util.*;

public class SortOddEven {
    public static void main(String[] args) {
        int[] nums = {5, 8, 11, 6, 2, 1, 7};
        int[] sortedArray = sortOddEven(nums);

        System.out.println("Sorted array with odd numbers in ascending and even numbers in
descending:");
        for (int num : sortedArray) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    public static int[] sortOddEven(int[] nums) {
        List<Integer> oddNumbers = new ArrayList<>();
        List<Integer> evenNumbers = new ArrayList<>();

        // Separate odd and even numbers into two separate lists
        for (int num : nums) {
            if (num % 2 == 0) {

```

```

        evenNumbers.add(num);
    } else {
        oddNumbers.add(num);
    }
}

// Sort odd numbers in ascending order
Collections.sort(oddNumbers);

// Sort even numbers in descending order
Collections.sort(evenNumbers, Collections.reverseOrder());

// Merge the sorted odd and even numbers
int[] sortedArray = new int[nums.length];
int i = 0;
for (int odd : oddNumbers) {
    sortedArray[i++] = odd;
}
for (int even : evenNumbers) {
    sortedArray[i++] = even;
}

return sortedArray;
}
}

```

62. It's about anagram. i/p was array of strings .and a word was given to find whether it has anagram in given array.

i/p catch, got, tiger, mat, eat, Pat, tap, tea

ANSWER ;

```
import java.util.Arrays;
```

```

public class AnagramFinder {
    public static void main(String[] args) {
        String[] words = {"catch", "got", "tiger", "mat", "eat", "Pat", "tap", "tea"};
        String wordToCheck = "chat";

        boolean hasAnagram = hasAnagram(words, wordToCheck);
        if (hasAnagram) {
            System.out.println("The word \"" + wordToCheck + "\" has an anagram in the array.");
        } else {
            System.out.println("The word \"" + wordToCheck + "\" does not have an anagram in the
array.");
        }
    }

    public static boolean hasAnagram(String[] words, String wordToCheck) {
        // Sort the characters of the word to check

```

```

char[] sortedWord = wordToCheck.toCharArray();
Arrays.sort(sortedWord);

// Iterate through each word in the array
for (String word : words) {
    // Skip the word if it has a different length
    if (word.length() != wordToCheck.length()) {
        continue;
    }

    // Sort the characters of the current word in the array
    char[] sortedCurrentWord = word.toCharArray();
    Arrays.sort(sortedCurrentWord);

    // Check if the sorted characters of both words are equal
    if (Arrays.equals(sortedWord, sortedCurrentWord)) {
        return true; // Found an anagram
    }
}

return false; // No anagram found
}
}

```

63.array of numbers were given to find a number which has same sum of numbers in it's either side.

I/p 1, 2, 3, 7, 6

O/p 7(has 1+ 2+3 in left 6 in right)

ANSWER;

```

public class EqualSumNumber {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 7, 6};
        int number = findEqualSumNumber(nums);

        if (number != -1) {
            System.out.println("Number with equal sum of numbers on its left and right sides: " +
number);
        } else {
            System.out.println("No such number found.");
        }
    }
}

public static int findEqualSumNumber(int[] nums) {
    int totalSum = 0;

    // Calculate total sum of all elements in the array
    for (int num : nums) {
        totalSum += num;
    }
}

```

```

    }

    int leftSum = 0;

    // Iterate through each element in the array
    for (int i = 0; i < nums.length; i++) {
        // Calculate sum of elements to the left of current element
        if (i > 0) {
            leftSum += nums[i - 1];
        }

        // Calculate sum of elements to the right of current element
        int rightSum = totalSum - leftSum - nums[i];

        // Check if sum of elements to the left and right are equal
        if (leftSum == rightSum) {
            return nums[i]; // Found the number with equal sum on both sides
        }
    }

    return -1; // No such number found
}
}

```

64.prime number – print n prime numbers
ANSWER;

```

public class PrimeNumbers {
    public static void main(String[] args) {
        int n = 10; // Number of prime numbers to print
        printFirstNPrimes(n);
    }

    public static void printFirstNPrimes(int n) {
        int count = 0;
        int num = 2; // Start with the first prime number

        System.out.println("First " + n + " prime numbers:");

        // Iterate until we find n prime numbers
        while (count < n) {
            if (isPrime(num)) {
                System.out.print(num + " ");
                count++;
            }
            num++;
        }
    }
}

```

```

public static boolean isPrime(int num) {
    if (num <= 1) {
        return false;
    }
    if (num <= 3) {
        return true;
    }
    if (num % 2 == 0 || num % 3 == 0) {
        return false;
    }

    // Check divisibility by numbers of the form 6k ± 1
    for (int i = 5; i * i <= num; i += 6) {
        if (num % i == 0 || num % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}
}

```

65.prime factor – sort the array based on the minimum factor they have.
ANSWER;

```
import java.util.*;
```

```

public class SortByMinPrimeFactor {
    public static void main(String[] args) {
        int[] nums = {15, 10, 9, 7, 8, 6};
        System.out.println("Original array: " + Arrays.toString(nums));

        Arrays.sort(nums, new MinPrimeFactorComparator());

        System.out.println("Sorted array by minimum prime factor: " + Arrays.toString(nums));
    }
}

```

```

static class MinPrimeFactorComparator implements Comparator<Integer> {
    @Override
    public int compare(Integer num1, Integer num2) {
        int minPrimeFactor1 = minPrimeFactor(num1);
        int minPrimeFactor2 = minPrimeFactor(num2);

        // Compare based on the minimum prime factor
        return Integer.compare(minPrimeFactor1, minPrimeFactor2);
    }
}

```

```

private int minPrimeFactor(int num) {
    if (num <= 1) {
        return Integer.MAX_VALUE; // No prime factor for 0 and 1
    }
}

```

```

    }

    // Find the minimum prime factor
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) {
            return i;
        }
    }

    return num; // If the number itself is prime, return the number
}
}
}

```

66.adding a digit to all the digits of a number eg digit=4, number = 2875, o/p= 612119
ANSWER;

```

public class AddDigitToNumber {
    public static void main(String[] args) {
        int digit = 4;
        int number = 2875;
        int result = addDigitToNumber(digit, number);
        System.out.println("Result: " + result); // Output: 612119
    }

    public static int addDigitToNumber(int digit, int number) {
        // Convert the number to a string
        String numberStr = String.valueOf(number);
        StringBuilder result = new StringBuilder();

        // Iterate through each character of the string representing the digits
        for (char c : numberStr.toCharArray()) {
            // Convert the character to an integer, add the given digit, and append to the result string
            result.append(Character.getNumericValue(c) + digit);
        }

        // Convert the result string back to an integer
        return Integer.parseInt(result.toString());
    }
}

```

67.form the largest possible number using the array of numbers.
ANSWER;

```

import java.util.*;

public class LargestNumber {
    public static void main(String[] args) {
        int[] nums = {3, 30, 34, 5, 9};
    }
}

```

```

    String largestNumber = largestNumber(nums);
    System.out.println("Largest number: " + largestNumber); // Output: 9534330
}

public static String largestNumber(int[] nums) {
    // Convert the array of numbers to an array of strings
    String[] numsStr = new String[nums.length];
    for (int i = 0; i < nums.length; i++) {
        numsStr[i] = String.valueOf(nums[i]);
    }

    // Define a custom comparator to compare two numbers based on their concatenated string
    // representation
    Comparator<String> comparator = new Comparator<String>() {
        @Override
        public int compare(String num1, String num2) {
            String order1 = num1 + num2;
            String order2 = num2 + num1;
            // Compare the two concatenated strings to determine their order
            return order2.compareTo(order1); // Reverse order to sort in descending order
        }
    };

    // Sort the array of strings using the custom comparator
    Arrays.sort(numsStr, comparator);

    // Handle edge case where all elements are 0
    if (numsStr[0].equals("0")) {
        return "0";
    }

    // Concatenate the sorted numbers to form the largest possible number
    StringBuilder result = new StringBuilder();
    for (String num : numsStr) {
        result.append(num);
    }

    return result.toString();
}
}

```

68.lexicographic sorting.
ANSWER;

```

import java.util.Arrays;

public class LexicographicSorting {
    public static void main(String[] args) {
        String[] words = {"banana", "apple", "orange", "grape", "cherry"};
    }
}

```

```

// Sort the array lexicographically
Arrays.sort(words);

// Print the sorted array
System.out.println("Sorted array:");
for (String word : words) {
    System.out.println(word);
}
}
}

```

69. given a set of numbers, and a digit in each iteration, if the digit exists in any of the numbers, remove its occurrences and ask for the next digit till the list becomes empty.

ANSWER;

```

import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class RemoveDigitsFromSet {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input set of numbers
        Set<Integer> numbers = new HashSet<>();
        numbers.add(12345);
        numbers.add(67890);
        numbers.add(54321);

        // Display the original set of numbers
        System.out.println("Original set of numbers: " + numbers);

        // Iterate through each digit
        System.out.println("Enter digits to remove (enter -1 to stop):");
        while (true) {
            int digit = scanner.nextInt();
            if (digit == -1) {
                break; // Exit the loop if -1 is entered
            }

            // Remove occurrences of the digit from the set of numbers
            removeDigitOccurrences(numbers, digit);

            // Display the updated set of numbers
            System.out.println("Updated set of numbers: " + numbers);
        }

        scanner.close();
    }
}

```



```

    }

    public static void removeDigitOccurrences(Set<Integer> numbers, int digit) {
        // Iterate through each number in the set
        for (Integer number : numbers) {
            // Convert the number to a string to check for digit occurrences
            String numberStr = String.valueOf(number);
            // Remove all occurrences of the digit from the string representation of the number
            numberStr = numberStr.replace(String.valueOf(digit), "");
            // Convert the modified string back to an integer and update the set
            numbers.remove(number);
            if (!numberStr.isEmpty()) {
                numbers.add(Integer.parseInt(numberStr));
            }
        }
    }
}

```

70. Check if a number 'a' is present in another number 'b'.
ANSWER;

```

public class NumberSubstring {
    public static void main(String[] args) {
        int a = 123;
        int b = 5123487;

        boolean result = isNumberPresent(a, b);

        if (result) {
            System.out.println(a + " is present in " + b);
        } else {
            System.out.println(a + " is not present in " + b);
        }
    }

    public static boolean isNumberPresent(int a, int b) {
        // Convert numbers to strings
        String aStr = String.valueOf(a);
        String bStr = String.valueOf(b);

        // Check if 'a' is a substring of 'b'
        return bStr.contains(aStr);
    }
}

```

71. Find the extra element and its index

Input : [10, 20, 30, 12, 5]

[10, 5, 30, 20]

Output : 12 is the extra element in array 1 at index 4

Input : [-1, 0, 3, 2]

[3, 4, 0, -1, 2]

Output : 4 is the extra element in array 3 at index 5

ANSWER;

```
import java.util.*;
```

```
public class ExtraElement {
    public static void main(String[] args) {
        int[] arr1 = {10, 20, 30, 12, 5};
        int[] arr2 = {10, 5, 30, 20};
        findExtraElementAndIndex(arr1, arr2);

        int[] arr3 = {-1, 0, 3, 2};
        int[] arr4 = {3, 4, 0, -1, 2};
        findExtraElementAndIndex(arr3, arr4);
    }

    public static void findExtraElementAndIndex(int[] arr1, int[] arr2) {
        Set<Integer> set = new HashSet<>();
        // Add all elements of arr2 to a set for efficient lookup
        for (int num : arr2) {
            set.add(num);
        }

        // Iterate through arr1 to find the extra element
        for (int i = 0; i < arr1.length; i++) {
            if (!set.contains(arr1[i])) {
                System.out.println(arr1[i] + " is the extra element in array 1 at index " + i);
                return;
            }
        }
        System.out.println("No extra element found in array 1.");
    }
}
```

72. Find the least prime number that can be added with first array element that makes them divisible by second array elements at respective index (check for prime numbers under 1000, if exist return -1 as answer) & (Consider 1 as prime number)

Input : [20, 7]

[11, 5]

Output : [1, 3]

Explanation :

(20 + ?) % 11

(7 + ?) % 5

ANSWER;

```

import java.util.*;

public class LeastPrimeAddition {
    public static void main(String[] args) {
        int[] arr1 = {20, 7};
        int[] arr2 = {11, 5};

        int[] result = findLeastPrimeAddition(arr1, arr2);
        System.out.println(Arrays.toString(result));
    }

    public static int[] findLeastPrimeAddition(int[] arr1, int[] arr2) {
        int[] result = new int[arr1.length];

        for (int i = 0; i < arr1.length; i++) {
            // Find the least prime number that, when added to arr1[i], makes it divisible by arr2[i]
            result[i] = leastPrimeToAdd(arr1[i], arr2[i]);
        }

        return result;
    }

    public static int leastPrimeToAdd(int num1, int num2) {
        // Iterate through prime numbers under 1000 to find the least prime number that satisfies the
        condition
        for (int prime = 1; prime < 1000; prime++) {
            if (isPrime(prime) && (num1 + prime) % num2 == 0) {
                return prime;
            }
        }
        // If no such prime number exists, return -1
        return -1;
    }

    public static boolean isPrime(int num) {
        if (num <= 1) {
            return false;
        }
        for (int i = 2; i * i <= num; i++) {
            if (num % i == 0) {
                return false;
            }
        }
        return true;
    }
}

```

73. Sort the array elements in descending order according to their frequency of occurrence

Input : [2 2 3 4 5 12 2 3 3 3 12]

Output : 3 3 3 3 2 2 2 12 12 4 5

Explanation : 3 occurred 4 times, 2 occurred 3 times, 12 occurred 2 times, 4 occurred 1 time, 5 occurred 1 time

Input : [0 -1 2 1 0]

Output : 0 0 -1 1 2

Note : sort single occurrence elements in ascending order

ANSWER;

```
import java.util.*;
```

```
public class SortByFrequency {
    public static void main(String[] args) {
        int[] arr1 = {2, 2, 3, 4, 5, 12, 2, 3, 3, 3, 12};
        int[] arr2 = {0, -1, 2, 1, 0};

        System.out.println("Input 1:");
        System.out.println(Arrays.toString(arr1));
        System.out.println("Output 1:");
        System.out.println(Arrays.toString(sortByFrequency(arr1)));

        System.out.println("\nInput 2:");
        System.out.println(Arrays.toString(arr2));
        System.out.println("Output 2:");
        System.out.println(Arrays.toString(sortByFrequency(arr2)));
    }

    public static int[] sortByFrequency(int[] arr) {
        // Count the frequency of each element
        Map<Integer, Integer> frequencyMap = new HashMap<>();
        for (int num : arr) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }

        // Sort the elements based on their frequencies
        Arrays.sort(arr, (a, b) -> {
            int freqComparison = frequencyMap.get(b).compareTo(frequencyMap.get(a));
            if (freqComparison != 0) {
                return freqComparison; // Sort by frequency in descending order
            } else {
                return a - b; // Sort by value in ascending order if frequencies are equal
            }
        });

        return arr;
    }
}
```

74. Print true if second string is a substring of first string, else print false.

Note : * symbol can replace n number of characters

Input : Spoon Sp*n Output : TRUE

Zoho *o*o Output : TRUE

Man n* Output : FALSE

Subline line Output : TRUE

ANSWER;

```
public class SubstringCheck {
    public static void main(String[] args) {
        System.out.println(isSubstring("Spoon", "Sp*n")); // Output: true
        System.out.println(isSubstring("Zoho", "**o*o")); // Output: true
        System.out.println(isSubstring("Man", "n*")); // Output: false
        System.out.println(isSubstring("Subline", "line")); // Output: true
    }

    public static boolean isSubstring(String str1, String str2) {
        // Replace '*' with a regular expression pattern matching any character zero or more times
        String pattern = str2.replace("*", ".");

        // Use contains() method to check if the pattern exists in str1
        return str1.matches(".*" + pattern + ".*");
    }
}
```

75. Print second frequently occurring number in given series

Example :

Input: 1 1 2 3 1 2 4

Output: 2

Explanation: 1 occurs 3 times, 2 occurs 2 times, 3 occurs 1 time and 4 occurs 1 time. Hence second frequently occurring number in given series is 2

ANSWER;

```
import java.util.*;

public class SecondMostFrequent {
    public static void main(String[] args) {
        int[] series = {1, 1, 2, 3, 1, 2, 4};
        int result = findSecondMostFrequent(series);
        System.out.println("Second most frequent number: " + result); // Output: 2
    }

    public static int findSecondMostFrequent(int[] series) {
        Map<Integer, Integer> frequencyMap = new HashMap<>();

        // Count the frequency of each number in the series
        for (int num : series) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
    }
}
```

```

// Create a list of map entries for sorting
List<Map.Entry<Integer, Integer>> entryList = new ArrayList<>(frequencyMap.entrySet());

// Sort the list based on frequency counts in descending order
entryList.sort((entry1, entry2) -> entry2.getValue().compareTo(entry1.getValue()));

// Get the second highest count
int secondHighestCount = entryList.get(1).getValue();

// Find the number corresponding to the second highest count
for (Map.Entry<Integer, Integer> entry : entryList) {
    if (entry.getValue() == secondHighestCount) {
        return entry.getKey();
    }
}

// If there is no second highest count, return -1
return -1;
}
}

```

76. Print only numbers which are present in Fibonacci series (0 1 1 2 3 5 8)

Example:

Input: 2 10 4 8

Output: 2 8

Input: 1 10 6 8 13 21

Output: 1 8 13 21

ANSWER;

```

import java.util.*;

public class FibonacciNumbers {
    public static void main(String[] args) {
        int[] input1 = {2, 10, 4, 8};
        System.out.println("Input 1: " + Arrays.toString(input1));
        System.out.println("Output 1: " + Arrays.toString(getFibonacciNumbers(input1)));

        int[] input2 = {1, 10, 6, 8, 13, 21};
        System.out.println("\nInput 2: " + Arrays.toString(input2));
        System.out.println("Output 2: " + Arrays.toString(getFibonacciNumbers(input2)));
    }

    public static List<Integer> generateFibonacciSeries(int limit) {
        List<Integer> fibonacciSeries = new ArrayList<>();
        int a = 0, b = 1;
        while (a <= limit) {
            fibonacciSeries.add(a);
            int temp = a + b;
            a = b;
        }
    }
}

```

```

        b = temp;
    }
    return fibonacciSeries;
}

public static int[] getFibonacciNumbers(int[] input) {
    // Generate Fibonacci series up to the maximum number in the input
    int maxInput = Arrays.stream(input).max().getAsInt();
    List<Integer> fibonacciSeries = generateFibonacciSeries(maxInput);

    // Filter numbers from input that are present in the Fibonacci series
    List<Integer> result = new ArrayList<>();
    for (int num : input) {
        if (fibonacciSeries.contains(num)) {
            result.add(num);
        }
    }

    // Convert List<Integer> to int[]
    return result.stream().mapToInt(Integer::intValue).toArray();
}
}

```

77.. Print pattern like this

Example:

Input: 1

Output: 0

Input: 2

Output:

0 0

0 1

1 0

1 1

Input: 3

Output:

0 0 0

0 0 1

0 1 0

0 1 1

1 0 0

1 0 1

1 1 0

1 1 1

ANSWER;

```
import java.util.*;
```

```

public class BinaryPattern {
    public static void main(String[] args) {
        int input1 = 1;
        System.out.println("Input: " + input1);
        System.out.println("Output:");
        printBinaryPattern(input1);

        int input2 = 2;
        System.out.println("\nInput: " + input2);
        System.out.println("Output:");
        printBinaryPattern(input2);

        int input3 = 3;
        System.out.println("\nInput: " + input3);
        System.out.println("Output:");
        printBinaryPattern(input3);
    }

    public static void printBinaryPattern(int n) {
        // Create an array to store the current combination
        int[] current = new int[n];
        // Start generating patterns
        generatePatterns(current, 0, n);
    }

    public static void generatePatterns(int[] current, int index, int n) {
        // If the current combination is complete, print it
        if (index == n) {
            for (int num : current) {
                System.out.print(num + " ");
            }
            System.out.println();
            return;
        }
        // Try 0 and 1 at the current index
        for (int i = 0; i <= 1; i++) {
            current[index] = i;
            generatePatterns(current, index + 1, n);
        }
    }
}

```

78. NxN matrix will be provided. 0->block, 1->Not a block

Always starting point is (0,0), Ending point is (N-1,N-1).

You have to go from starting point to ending point. One valid solution is enough.

Example:

Input:

N=4


```

1 1 0 0
1 0 0 1
1 1 1 1
0 0 0 1
Output:
_ 1 0 0
_ 0 0 1

_ _ _ _
0 0 0 _
ANSWER;

```

```

import java.util.*;

public class Pathfinder {
    public static void main(String[] args) {
        int[][] grid = {
            {1, 1, 0, 0},
            {1, 0, 0, 1},
            {1, 1, 1, 1},
            {0, 0, 0, 1}
        };
        int N = grid.length;

        System.out.println("Input:");
        printMatrix(grid);

        System.out.println("\nOutput:");
        findPath(grid, N, 0, 0);
    }

    public static void findPath(int[][] grid, int N, int row, int col) {
        // Base case: if current position is out of bounds or blocked, return
        if (row < 0 || col < 0 || row >= N || col >= N || grid[row][col] == 0) {
            return;
        }

        // Mark current position as visited
        grid[row][col] = -1;

        // If reached the destination, print the path
        if (row == N - 1 && col == N - 1) {
            printMatrix(grid);
            return;
        }

        // Explore all four directions recursively
        findPath(grid, N, row + 1, col); // Down
        findPath(grid, N, row - 1, col); // Up
        findPath(grid, N, row, col + 1); // Right
    }
}

```

```

        findPath(grid, N, row, col - 1); // Left

        // Backtrack: mark current position as unvisited
        grid[row][col] = 0;
    }

    public static void printMatrix(int[][] grid) {
        for (int[] row : grid) {
            for (int cell : row) {
                System.out.print(cell == -1 ? "_ " : cell + " ");
            }
            System.out.println();
        }
    }
}

```

79.. Insert 0 after consecutive (K times) of 1 is found.

Example:

Input:

Number of bits: 12

Bits: 1 0 1 1 0 1 1 0 1 1 1 1

Consecutive K: 2

Output:

1 0 1 1 0 0 1 1 0 0 1 1 0 1 1 0

ANSWER;

```

import java.util.*;

public class InsertZeros {
    public static void main(String[] args) {
        int[] bits = {1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1};
        int k = 2;

        System.out.println("Input:");
        System.out.println("Number of bits: " + bits.length);
        System.out.println("Bits: " + Arrays.toString(bits));
        System.out.println("Consecutive K: " + k);

        int[] result = insertZeros(bits, k);

        System.out.println("\nOutput:");
        System.out.println(Arrays.toString(result));
    }

    public static int[] insertZeros(int[] bits, int k) {
        List<Integer> resultList = new ArrayList<>();
        int consecutiveOnes = 0;
    }
}

```

```

for (int bit : bits) {
    resultList.add(bit);
    if (bit == 1) {
        consecutiveOnes++;
        if (consecutiveOnes == k) {
            resultList.add(0); // Insert 0 after K consecutive 1s
            consecutiveOnes = 0; // Reset consecutiveOnes counter
        }
    } else {
        consecutiveOnes = 0; // Reset consecutiveOnes counter
    }
}

// Convert List<Integer> to int[]
int[] result = new int[resultList.size()];
for (int i = 0; i < resultList.size(); i++) {
    result[i] = resultList.get(i);
}

return result;
}
}

```

80. Find the maximum of three numbers?

ANSWER;

```

public class MaxOfThreeNumbers {
    public static void main(String[] args) {
        int num1 = 10;
        int num2 = 20;
        int num3 = 15;

        int max = findMax(num1, num2, num3);
        System.out.println("The maximum of " + num1 + ", " + num2 + ", and " + num3 + " is: " +
max);
    }

    public static int findMax(int a, int b, int c) {
        int maxAB = Math.max(a, b); // Maximum of first two numbers
        return Math.max(maxAB, c); // Maximum of the maximum of first two numbers and the third
number
    }
}

```

81. Print the total number of odd and even digits in the given number.

Ex. Input : 1234567

Output : ODD 4

EVEN 3

ANSWER;

```
public class CountOddEvenDigits {
    public static void main(String[] args) {
        int number = 1234567;
        countOddEvenDigits(number);
    }

    public static void countOddEvenDigits(int number) {
        int oddCount = 0;
        int evenCount = 0;

        // Iterate through each digit of the number
        while (number > 0) {
            int digit = number % 10;
            if (digit % 2 == 0) {
                evenCount++;
            } else {
                oddCount++;
            }
            number /= 10;
        }

        System.out.println("ODD " + oddCount);
        System.out.println("EVEN " + evenCount);
    }
}
```

82. Find the second maximum among the given numbers.

Ex. INPUT :

Size of Array : 8

Enter the elements : 2 5 1 6 2 6 7 10

OUTPUT :

7

Ex. INPUT

Size of Array : 4

Enter the elements : 4 1 2 2

OUTPUT :2

Ex. INPUT :

Size of Array : 1

Enter the elements : 1

OUTPUT :

No second maximum

ANSWER;

```
import java.util.Scanner;

public class SecondMaximum {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Size of Array: ");
        int size = scanner.nextInt();

        if (size <= 1) {
            System.out.println("No second maximum");
            return;
        }

        int[] arr = new int[size];
        System.out.print("Enter the elements: ");
        for (int i = 0; i < size; i++) {
            arr[i] = scanner.nextInt();
        }

        int secondMax = findSecondMaximum(arr);
        if (secondMax == Integer.MIN_VALUE) {
            System.out.println("No second maximum");
        } else {
            System.out.println("Second maximum: " + secondMax);
        }

        scanner.close();
    }

    public static int findSecondMaximum(int[] arr) {
        int max = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > max) {
                secondMax = max;
                max = num;
            } else if (num > secondMax && num != max) {
                secondMax = num;
            }
        }
    }
}
```

```

        return secondMax;
    }
}

```

83. Print the following pattern

Ex. INPUT : 5

OUTPUT :

```

    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1

```

ANSWER;

```

import java.util.Scanner;

public class PascalTrianglePattern {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of rows: ");
        int rows = scanner.nextInt();

        printPascalTriangle(rows);

        scanner.close();
    }

    public static void printPascalTriangle(int rows) {
        for (int i = 0; i < rows; i++) {
            // Print spaces for alignment
            for (int j = 0; j < rows - i - 1; j++) {
                System.out.print(" ");
            }

            // Print values in each row
            for (int j = 0; j <= i; j++) {
                System.out.print(binomialCoefficient(i, j) + " ");
            }

            System.out.println();
        }
    }
}

```

```

public static int binomialCoefficient(int n, int k) {
    return factorial(n) / (factorial(k) * factorial(n - k));
}

public static int factorial(int n) {
    if (n == 0) {
        return 1;
    }

    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }

    return result;
}
}

```

84. Given a two dimensional array which consists of only 0's and 1's. Print the matrix without duplication.

Ex. INPUT :

```

Enter Row Size : 4
Enter column size : 3
Enter the matrix :
1 0 1
1 1 0
1 1 1
1 0 1

```

OUTPUT :

```

Unique Matrix :
1 0 1
1 1 0
1 1 1

```

ANSWER;

```

import java.util.*;

public class UniqueMatrix {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter Row Size: ");
    }
}

```

```

int rows = scanner.nextInt();
System.out.print("Enter Column Size: ");
int columns = scanner.nextInt();

int[][] matrix = new int[rows][columns];
System.out.println("Enter the matrix:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < columns; j++) {
        matrix[i][j] = scanner.nextInt();
    }
}

System.out.println("\nUnique Matrix:");
printUniqueMatrix(matrix);

scanner.close();
}

public static void printUniqueMatrix(int[][] matrix) {
    Set<String> uniqueRows = new HashSet<>();

    for (int[] row : matrix) {
        String rowString = Arrays.toString(row);
        if (!uniqueRows.contains(rowString)) {
            uniqueRows.add(rowString);
            System.out.println(Arrays.toString(row).replaceAll("[\\[\\],]", ""));
        }
    }
}
}

```

85. Given an array of positive numbers. Print the numbers which have longest continuous range.

Ex. INPUT :

Enter array size : 8

Enter array elements : 1 3 10 7 9 2 4

OUTPUT :

1 2 3 4

Ex. INPUT :

Enter array size : 8

Enter array elements : 1 3 9 7 8 2 4 6

OUTPUT :

1 2 3 4

6 7 8 9

ANSWER;

```
import java.util.*;

public class LongestContinuousRange {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int size = scanner.nextInt();

        int[] array = new int[size];
        System.out.print("Enter array elements: ");
        for (int i = 0; i < size; i++) {
            array[i] = scanner.nextInt();
        }

        printLongestContinuousRange(array);

        scanner.close();
    }

    public static void printLongestContinuousRange(int[] array) {
        Arrays.sort(array);

        int maxLength = 1;
        int currentLength = 1;
        int startIdx = 0;
        int endIdx = 0;

        for (int i = 1; i < array.length; i++) {
            if (array[i] == array[i - 1] + 1) {
                currentLength++;
            } else {
                if (currentLength > maxLength) {
                    maxLength = currentLength;
                    startIdx = i - maxLength;
                    endIdx = i - 1;
                }
                currentLength = 1;
            }
        }

        if (currentLength > maxLength) {
            maxLength = currentLength;
        }
    }
}
```

```

        startIdx = array.length - maxLength;
        endIdx = array.length - 1;
    }

    System.out.println("Longest continuous range(s):");
    for (int i = startIdx; i <= endIdx; i++) {
        System.out.print(array[i] + " ");
    }
    System.out.println();
}
}

```

86. Given two arrays. Find its union.

Input :

Enter size of first array : 6

Enter the elements : 1 2 3 4 5 3

Enter size of second array : 4

Enter the elements : 1 2 7 5

OUTPUT :

1 2 3 4 5 7

ANSWER;

```

import java.util.*;

public class ArrayUnion {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input for the first array
        System.out.print("Enter size of first array: ");
        int size1 = scanner.nextInt();
        int[] array1 = new int[size1];
        System.out.print("Enter the elements: ");
        for (int i = 0; i < size1; i++) {
            array1[i] = scanner.nextInt();
        }

        // Input for the second array
        System.out.print("Enter size of second array: ");
        int size2 = scanner.nextInt();
        int[] array2 = new int[size2];
    }
}

```

```

        System.out.print("Enter the elements: ");
        for (int i = 0; i < size2; i++) {
            array2[i] = scanner.nextInt();
        }

        // Find union and print the result
        System.out.println("Output:");
        printUnion(array1, array2);

        scanner.close();
    }

    public static void printUnion(int[] arr1, int[] arr2) {
        Set<Integer> unionSet = new HashSet<>();

        // Add elements from the first array to the union set
        for (int num : arr1) {
            unionSet.add(num);
        }

        // Add elements from the second array to the union set
        for (int num : arr2) {
            unionSet.add(num);
        }

        // Print the union set
        for (int num : unionSet) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}

```

87. Given an array of numbers. Print the numbers without duplication.

INPUT :

Enter the array size : 4

Enter the elements : 1 1 2 4

OUTPUT :

1 2 4

ANSWER;

```
import java.util.*;

public class RemoveDuplicatesFromArray {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the array size: ");
        int size = scanner.nextInt();

        int[] array = new int[size];
        System.out.print("Enter the elements: ");
        for (int i = 0; i < size; i++) {
            array[i] = scanner.nextInt();
        }

        System.out.println("Output:");
        printArrayWithoutDuplicates(array);

        scanner.close();
    }

    public static void printArrayWithoutDuplicates(int[] array) {
        Set<Integer> uniqueSet = new HashSet<>();

        for (int num : array) {
            uniqueSet.add(num);
        }

        for (int num : uniqueSet) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

88. Given an array of numbers and a number k. Print the maximum possible k digit number which can be formed using given numbers.

INPUT :

Enter the array size : 4

Enter the elements : 1 4 973 97

Enter number of digits : 3

OUTPUT :

974

INPUT :

Enter the array size : 6

Enter the elements : 1 4 89 73 9 7

Enter number of digits : 5

OUTPUT :

98973

ANSWER;

```
import java.util.*;

public class MaxKDigitNumber {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the array size: ");
        int size = scanner.nextInt();

        int[] array = new int[size];
        System.out.print("Enter the elements: ");
        for (int i = 0; i < size; i++) {
            array[i] = scanner.nextInt();
        }

        System.out.print("Enter number of digits: ");
        int k = scanner.nextInt();

        System.out.println("Output:");
        System.out.println(getMaxKDigitNumber(array, k));

        scanner.close();
    }

    public static String getMaxKDigitNumber(int[] array, int k) {
        Arrays.sort(array);
        StringBuilder result = new StringBuilder();

        for (int i = array.length - 1; i >= 0; i--) {
            result.append(array[i]);
        }
    }
}
```

```

        if (result.length() == k) {
            break;
        }
    }

    return result.toString();
}
}

```

89. Given an array of numbers and a window of size k. Print the maximum of numbers inside the window for each step as the window moves from the beginning of the array.

INPUT :

Enter the array size : 8

Enter the elements : 1,3,5,2,1,8,6,9

Enter the window size : 3

OUTPUT :

5 5 5 8 8 9

ANSWER;

```

import java.util.*;

public class SlidingWindowMaximum {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the array size: ");
        int size = scanner.nextInt();

        int[] array = new int[size];
        System.out.print("Enter the elements: ");
        for (int i = 0; i < size; i++) {
            array[i] = scanner.nextInt();
        }

        System.out.print("Enter the window size: ");
        int k = scanner.nextInt();

        System.out.println("Output:");
        printSlidingWindowMax(array, k);

        scanner.close();
    }
}

```

```

public static void printSlidingWindowMax(int[] array, int k) {
    Deque<Integer> deque = new LinkedList<>();

    // Process the first window
    for (int i = 0; i < k; i++) {
        while (!deque.isEmpty() && array[i] >= array[deque.peekLast()]) {
            deque.removeLast();
        }
        deque.addLast(i);
    }
    System.out.print(array[deque.peekFirst()] + " ");

    // Process the rest of the windows
    for (int i = k; i < array.length; i++) {
        while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
            deque.removeFirst();
        }
        while (!deque.isEmpty() && array[i] >= array[deque.peekLast()]) {
            deque.removeLast();
        }
        deque.addLast(i);
        System.out.print(array[deque.peekFirst()] + " ");
    }
    System.out.println();
}
}

```

90: Given a string, reverse only vowels in it; leaving rest of the string as it is.

Input : abcdef

Output : ebcdaf

ANSWER;

```

import java.util.Scanner;

public class ReverseVowels {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        String output = reverseVowels(input);
        System.out.println("Output: " + output);

        scanner.close();
    }
}

```

```

    }

    public static String reverseVowels(String s) {
        char[] charArray = s.toCharArray();
        int left = 0, right = charArray.length - 1;

        while (left < right) {
            // Find the next vowel from the left
            while (left < right && !isVowel(charArray[left])) {
                left++;
            }
            // Find the next vowel from the right
            while (left < right && !isVowel(charArray[right])) {
                right--;
            }
            // Swap the vowels
            if (left < right) {
                char temp = charArray[left];
                charArray[left] = charArray[right];
                charArray[right] = temp;
                left++;
                right--;
            }
        }

        return new String(charArray);
    }

    public static boolean isVowel(char c) {
        c = Character.toLowerCase(c);
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
    }
}

```

91 : Write a program to check if the given words are present in matrix given below. The words can be left to right, top to bottom and the diagonals (in top to bottom direction)

ANSWER;

```

import java.util.*;

public class WordSearch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}

```



```

// Define the matrix
char[][] matrix = {
    {'A', 'B', 'C', 'D'},
    {'E', 'F', 'G', 'H'},
    {'I', 'J', 'K', 'L'},
    {'M', 'N', 'O', 'P'}
};

// Input the words to search for
System.out.print("Enter the number of words to search for: ");
int numWords = scanner.nextInt();
scanner.nextLine(); // Consume newline
String[] words = new String[numWords];
for (int i = 0; i < numWords; i++) {
    System.out.print("Enter word " + (i + 1) + ": ");
    words[i] = scanner.nextLine();
}

// Check if each word is present in the matrix
for (String word : words) {
    if (searchWord(matrix, word)) {
        System.out.println("Word " + word + " found.");
    } else {
        System.out.println("Word " + word + " not found.");
    }
}

scanner.close();
}

public static boolean searchWord(char[][] matrix, String word) {
    int rows = matrix.length;
    int cols = matrix[0].length;

    // Search left to right
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j <= cols - word.length(); j++) {
            if (checkWord(matrix, i, j, word, 0, 1)) {
                return true;
            }
        }
    }
}

```

```

// Search top to bottom
for (int i = 0; i <= rows - word.length(); i++) {
    for (int j = 0; j < cols; j++) {
        if (checkWord(matrix, i, j, word, 1, 0)) {
            return true;
        }
    }
}

// Search diagonally (top-left to bottom-right)
for (int i = 0; i <= rows - word.length(); i++) {
    for (int j = 0; j <= cols - word.length(); j++) {
        if (checkWord(matrix, i, j, word, 1, 1)) {
            return true;
        }
    }
}

return false;
}

public static boolean checkWord(char[][] matrix, int row, int col, String word, int dx, int dy) {
    int rows = matrix.length;
    int cols = matrix[0].length;

    for (int i = 0; i < word.length(); i++) {
        if (matrix[row][col] != word.charAt(i)) {
            return false;
        }
        row += dx;
        col += dy;
        if (row < 0 || row >= rows || col < 0 || col >= cols) {
            return false;
        }
    }
    return true;
}
}

```

92 : Write a program to form lines using given set of words. The line formation should follow below rules.

- i) Total characters in a single line excluding the space between the words and the favorite character should not exceed the given number.**
- ii) Favorite character is case insensitive.**

iii) Words should not be broken up. Complete words alone should be used in a single line. A word should be used in one line only.

Input : Max char per line = 10

Favorite character = 'o'

Words : Zoho, Eating, Watching, Pogo

Loving, Mango

Output : Watching Zoho

Eating Mango

Loving Pogo.

ANSWER;

```
import java.util.*;
```

```
public class LineFormation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input maximum characters per line and favorite character
        System.out.print("Enter maximum characters per line: ");
        int maxCharsPerLine = scanner.nextInt();
        scanner.nextLine(); // Consume newline
        System.out.print("Enter favorite character: ");
        char favoriteChar = scanner.nextLine().charAt(0);
        favoriteChar = Character.toLowerCase(favoriteChar); // Convert to lowercase

        // Input words
        System.out.print("Enter words separated by commas: ");
        String[] words = scanner.nextLine().split(",\\s*");

        // Form lines
        List<String> lines = formLines(maxCharsPerLine, favoriteChar, words);

        // Output lines
        System.out.println("Output:");
        for (String line : lines) {
            System.out.println(line);
        }

        scanner.close();
    }
}
```

```

public static List<String> formLines(int maxCharsPerLine, char favoriteChar, String[] words) {
    List<String> lines = new ArrayList<>();
    StringBuilder lineBuilder = new StringBuilder();
    int charsInLine = 0;

    for (String word : words) {
        // If adding the word to the current line exceeds maxCharsPerLine, start a new line
        if (charsInLine + word.length() + (lineBuilder.length() > 0 ? 1 : 0) > maxCharsPerLine) {
            lines.add(lineBuilder.toString());
            lineBuilder.setLength(0); // Clear the lineBuilder
            charsInLine = 0;
        }

        // If word contains favorite character, append it to the line and update charsInLine
        if (word.toLowerCase().indexOf(favoriteChar) != -1) {
            if (lineBuilder.length() > 0) {
                lineBuilder.append(" "); // Add space if appending to existing line
            }
            lineBuilder.append(word);
            charsInLine += word.length();
        }
    }

    // Add the last line if not empty
    if (lineBuilder.length() > 0) {
        lines.add(lineBuilder.toString());
    }

    return lines;
}

```

93. Adding 2 numbers

Given 2 huge numbers as separate digits, store them in array and process them and calculate the sum of 2 numbers and store the result in an array and print the sum.

Input:

Number of digits:12

9 2 8 1 3 5 6 7 3 1 1 6

Number of digits:9

7 8 4 6 2 1 9 9 7

Output :

9 2 8 9 2 0 2 9 5 1 1 3

ANSWER;

```

import java.util.Scanner;

public class HugeNumberAddition {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Input the number of digits for the first number
        System.out.print("Number of digits for the first number: ");
        int size1 = scanner.nextInt();
        int[] num1 = new int[size1];
        System.out.print("Enter the digits of the first number separated by space: ");
        for (int i = size1 - 1; i >= 0; i--) {
            num1[i] = scanner.nextInt();
        }

        // Input the number of digits for the second number
        System.out.print("Number of digits for the second number: ");
        int size2 = scanner.nextInt();
        int[] num2 = new int[size2];
        System.out.print("Enter the digits of the second number separated by space: ");
        for (int i = size2 - 1; i >= 0; i--) {
            num2[i] = scanner.nextInt();
        }

        // Calculate the sum
        int[] sum = addNumbers(num1, num2);

        // Output the sum
        System.out.println("Output:");
        for (int digit : sum) {
            System.out.print(digit + " ");
        }

        scanner.close();
    }

    public static int[] addNumbers(int[] num1, int[] num2) {
        int maxSize = Math.max(num1.length, num2.length);
        int[] sum = new int[maxSize + 1]; // To accommodate carry if needed

        int carry = 0;
        for (int i = 0; i < maxSize; i++) {
            int digit1 = (i < num1.length) ? num1[i] : 0;
            int digit2 = (i < num2.length) ? num2[i] : 0;

```

```

        int digitSum = digit1 + digit2 + carry;
        sum[i] = digitSum % 10; // Store the least significant digit
        carry = digitSum / 10; // Calculate carry for the next iteration
    }

    if (carry > 0) {
        sum[maxSize] = carry; // Store the carry in the last position
    }

    return reverseArray(sum);
}

public static int[] reverseArray(int[] array) {
    int[] reversedArray = new int[array.length];
    for (int i = 0; i < array.length; i++) {
        reversedArray[i] = array[array.length - 1 - i];
    }
    return reversedArray;
}
}

```

94. Given sorted array check if two numbers sum in it is a given value

Input

Array = {1 3 4 8 10 } N = 7

output

true

ANSWER;

```

public class TwoSum {
    public static void main(String[] args) {
        int[] array = {1, 3, 4, 8, 10};
        int target = 7;
        System.out.println("Output: " + hasTwoSum(array, target));
    }

    public static boolean hasTwoSum(int[] array, int target) {
        int left = 0;
        int right = array.length - 1;

        while (left < right) {
            int sum = array[left] + array[right];
            if (sum == target) {
                return true;
            }
        }
    }
}

```

```

    } else if (sum < target) {
        left++; // Move left pointer to increase sum
    } else {
        right--; // Move right pointer to decrease sum
    }
}

return false; // No pair found
}
}

```

95. Computing value of sin (x)

Input x = 30 n = 10

output = 0.5

Hint : The equation $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$

ANSWER;

```

public class SinCalculator {
    public static void main(String[] args) {
        double x = 30; // Angle in degrees
        int n = 10; // Number of terms in the series

        double radians = Math.toRadians(x); // Convert angle to radians
        double result = computeSin(radians, n);

        System.out.println("Output: " + result);
    }

    public static double computeSin(double x, int n) {
        double sinValue = 0;

        for (int i = 0; i < n; i++) {
            double term = Math.pow(-1, i) * Math.pow(x, 2 * i + 1) / factorial(2 * i + 1);
            sinValue += term;
        }

        return sinValue;
    }

    public static double factorial(int n) {
        double fact = 1;
        for (int i = 2; i <= n; i++) {
            fact *= i;
        }
    }
}

```

```

        return fact;
    }
}

```

96. Write function to find multiplication of 2 numbers using + operator You must use minimum possible iterations.

Input: 3 , 4

Output 12

ANSWER;

```

public class MultiplicationUsingAddition {
    public static void main(String[] args) {
        int a = 3;
        int b = 4;
        System.out.println("Output: " + multiply(a, b));
    }

    public static int multiply(int a, int b) {
        if (b == 0) {
            return 0;
        }
        if (b > 0) {
            return a + multiply(a, b - 1);
        } else {
            return -multiply(a, -b);
        }
    }
}

```

97. Given array find maximum sum of contiguous sub array

{-2 -3 4 -1 -2 1 5 -3}

output 7 elements [4 -1 -2 1 5]

ANSWER;

```

public class MaximumSubarraySum {
    public static void main(String[] args) {
        int[] nums = {-2, -3, 4, -1, -2, 1, 5, -3};
        int[] maxSubarray = findMaxSubarray(nums);

        System.out.println("Output:");
        System.out.print("Elements: ");
        for (int num : maxSubarray) {
            System.out.print(num + " ");
        }
    }
}

```



```
    System.out.println();
    System.out.println("Sum: " + sum(maxSubarray));
}
```

```
public static int[] findMaxSubarray(int[] nums) {
    int maxSum = nums[0];
    int currentSum = nums[0];
    int start = 0;
    int end = 0;
    int tempStart = 0;

    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > currentSum + nums[i]) {
            currentSum = nums[i];
            tempStart = i;
        } else {
            currentSum += nums[i];
        }

        if (currentSum > maxSum) {
            maxSum = currentSum;
            start = tempStart;
            end = i;
        }
    }

    int[] maxSubarray = new int[end - start + 1];
    for (int i = start, j = 0; i <= end; i++, j++) {
        maxSubarray[j] = nums[i];
    }

    return maxSubarray;
}
```

```
public static int sum(int[] nums) {
    int sum = 0;
    for (int num : nums) {
        sum += num;
    }
    return sum;
}
}
```

98. Given unsorted array find all combination of the element for a given sum. Order should be maintained.

Input :

8 3 4 7 9 N=7

Output

{3 4 } {7}

ANSWER;

```
import java.util.ArrayList;
import java.util.List;

public class CombinationSumWithOrder {
    public static void main(String[] args) {
        int[] nums = {8, 3, 4, 7, 9};
        int target = 7;
        List<List<Integer>> result = combinationSum(nums, target);

        System.out.println("Output:");
        for (List<Integer> combination : result) {
            System.out.print("{ ");
            for (int num : combination) {
                System.out.print(num + " ");
            }
            System.out.println("}");
        }
    }

    public static List<List<Integer>> combinationSum(int[] nums, int target) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, target, 0, new ArrayList<>(), result);
        return result;
    }

    public static void backtrack(int[] nums, int target, int start, List<Integer> current,
List<List<Integer>> result) {
        if (target == 0) {
            result.add(new ArrayList<>(current));
            return;
        }
        if (target < 0 || start == nums.length) {
            return;
        }

        for (int i = start; i < nums.length; i++) {
```

```

        current.add(nums[i]);
        backtrack(nums, target - nums[i], i + 1, current, result);
        current.remove(current.size() - 1);
    }
}
}

```

99. Given an odd length word which should be printed from the middle of the word. The output should be in the following pattern.

Example:

Input: PROGRAM

Output:

```

    G
  GR
 GRA
GRAM
GRAMP
GRAMPR
GRAMPRO
ANSWER;

```

```
import java.util.Scanner;
```

```

public class WordPattern {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Input: ");
        String word = scanner.nextLine();

        printPattern(word);

        scanner.close();
    }

    public static void printPattern(String word) {
        int length = word.length();
        int middleIndex = length / 2;

        for (int i = middleIndex; i >= 0; i--) {
            for (int j = 0; j < length; j++) {
                if (j >= middleIndex - i && j <= middleIndex + i) {
                    System.out.print(word.charAt(j));
                } else {
                    System.out.print(" ");
                }
            }
        }
    }
}

```

```

    }
}
    System.out.println();
}
}
}

```

100. It is a program to implement Least Recently Used (LRU) concept. Given a key, if it is already existed then it should be marked as recently used otherwise a value should be stored which is given as input and marked as recently used. The capacity is to store only 10 key, value pairs. If the table is full and given a new key; the key, value pair which is not recently used should be deleted which gives feasibility to store the new key, value pair.

ANSWER;

```

import java.util.LinkedHashMap;
import java.util.Map;

public class LRUCache {
    private final int capacity;
    private final LinkedHashMap<Integer, Integer> cache;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
                return size() > capacity;
            }
        };
    }

    public int get(int key) {
        return cache.containsKey(key) ? cache.get(key) : -1;
    }

    public void put(int key, int value) {
        cache.put(key, value);
    }

    public static void main(String[] args) {
        LRUCache cache = new LRUCache(3); // Capacity is 3 for demonstration

        // Example usage
    }
}

```

```

        cache.put(1, 1);
        cache.put(2, 2);
        cache.put(3, 3);

        System.out.println(cache.get(1)); // Output: 1 (Key 1 is recently used)

        cache.put(4, 4); // Key 2 will be removed as it's least recently used

        System.out.println(cache.get(2)); // Output: -1 (Key 2 is removed)
    }
}

```

101. Given a few pairs of names in the order child, father. The input is a person name and level number. The output should be the number of children in that particular level for the person given.

Example:

Input:

```

[
{Ram, Syam},
{Akil, Syam},
{Nikil, Ram},
{Subhash, Ram},
{Karthik, Akil}
];
Syam 2

```

Output: 3 (Syam has Ram and Akil in level 1 and in level 2 he have Nikil, Subhash, Karthik. So the answer is 3).

101 Given an array of positive integers. The output should be the number of occurrences of each number.

Example:

Input: {2, 3, 2, 6, 1, 6, 2}

Output:

1 – 1

2 – 3

3 – 1

6 – 2

ANSWER;

```
import java.util.*;
```

```

public class ChildCounter {
    static Map<String, List<String>> tree = new HashMap<>();

    public static void main(String[] args) {

```

```

List<String[]> pairs = new ArrayList<>();
pairs.add(new String[]{"Ram", "Syam"});
pairs.add(new String[]{"Akil", "Syam"});
pairs.add(new String[]{"Nikil", "Ram"});
pairs.add(new String[]{"Subhash", "Ram"});
pairs.add(new String[]{"Karthik", "Akil"});

for (String[] pair : pairs) {
    String child = pair[0];
    String father = pair[1];
    tree.putIfAbsent(father, new ArrayList<>());
    tree.get(father).add(child);
}

System.out.println(countChildren("Syam", 2)); // Output: 3
}

public static int countChildren(String person, int level) {
    return countChildrenDFS(person, level, 0);
}

private static int countChildrenDFS(String person, int targetLevel, int currentLevel) {
    if (currentLevel == targetLevel) {
        return tree.containsKey(person) ? tree.get(person).size() : 0;
    }

    int count = 0;
    if (tree.containsKey(person)) {
        for (String child : tree.get(person)) {
            count += countChildrenDFS(child, targetLevel, currentLevel + 1);
        }
    }
    return count;
}
}

import java.util.*;

public class NumberCounter {
    public static void main(String[] args) {
        int[] numbers = {2, 3, 2, 6, 1, 6, 2};
        Map<Integer, Integer> countMap = new HashMap<>();

        for (int num : numbers) {

```

```

        countMap.put(num, countMap.getOrDefault(num, 0) + 1);
    }

    System.out.println("Output:");
    for (Map.Entry<Integer, Integer> entry : countMap.entrySet()) {
        System.out.println(entry.getKey() + " - " + entry.getValue());
    }
}
}

```

102) Given an array, find the minimum of all the greater numbers for each element in the array.

Sample:

Array : {2, 3, 7, -1, 8, 5, 11}

Output:

{2->3, 3->5, 7->8, -1->2, 8->11, 5->7, 11->}

ANSWER;

```

import java.util.*;

public class MinimumOfGreaterNumbers {
    public static void main(String[] args) {
        int[] array = {2, 3, 7, -1, 8, 5, 11};
        Map<Integer, Integer> result = findMinimumOfGreaterNumbers(array);
        System.out.println("Output:");
        for (Map.Entry<Integer, Integer> entry : result.entrySet()) {
            System.out.println(entry.getKey() + "->" + entry.getValue());
        }
    }

    public static Map<Integer, Integer> findMinimumOfGreaterNumbers(int[] array) {
        TreeMap<Integer, Integer> map = new TreeMap<>();
        Map<Integer, Integer> result = new HashMap<>();

        for (int num : array) {
            Integer nextGreater = map.higherKey(num);
            if (nextGreater != null) {
                result.put(num, nextGreater);
            } else {
                result.put(num, null);
            }
            map.put(num, num);
        }
    }
}

```

```

    }

    return result;
}
}

```

103) Find the largest sum contiguous subarray which should not have negative numbers. We have to print the sum and the corresponding array elements which brought up the sum.

Sample:

Array : {-2, 7, 5, -1, 3, 2, 9, -7}

Output:

Sum : 14

Elements : 3, 2, 9

ANSWER;

```

public class LargestSumSubarrayWithoutNegatives {
    public static void main(String[] args) {
        int[] array = {-2, 7, 5, -1, 3, 2, 9, -7};
        Result result = findLargestSumSubarray(array);
        System.out.println("Output:");
        System.out.println("Sum: " + result.sum);
        System.out.print("Elements: ");
        for (int i = result.start; i <= result.end; i++) {
            System.out.print(array[i] + " ");
        }
        System.out.println();
    }

    public static Result findLargestSumSubarray(int[] array) {
        int maxEndingHere = 0;
        int maxSoFar = 0;
        int start = 0;
        int end = 0;
        int tempStart = 0;

        for (int i = 0; i < array.length; i++) {
            maxEndingHere += array[i];

            if (maxEndingHere < 0) {
                maxEndingHere = 0;
                tempStart = i + 1;
            }
        }
    }
}

```



```

        if (maxEndingHere > maxSoFar) {
            maxSoFar = maxEndingHere;
            start = tempStart;
            end = i;
        }
    }

    return new Result(maxSoFar, start, end);
}

static class Result {
    int sum;
    int start;
    int end;

    public Result(int sum, int start, int end) {
        this.sum = sum;
        this.start = start;
        this.end = end;
    }
}

```

104) Given a string, we have to reverse the string without changing the position of punctuations and spaces.

Sample: house no : 123@ cbe

Output: ebc32 1o : nes@ uoh

ANSWER;

```

public class ReverseStringWithPunctuation {
    public static void main(String[] args) {
        String input = "house no : 123@ cbe";
        String reversed = reverseStringWithPunctuation(input);
        System.out.println("Output: " + reversed);
    }

    public static String reverseStringWithPunctuation(String input) {
        char[] chars = input.toCharArray();
        int left = 0, right = chars.length - 1;

        while (left < right) {
            if (!Character.isAlphabetic(chars[left])) {

```

```

        left++;
    } else if (!Character.isAlphabetic(chars[right])) {
        right--;
    } else {
        swap(chars, left++, right--);
    }
}

return new String(chars);
}

private static void swap(char[] chars, int i, int j) {
    char temp = chars[i];
    chars[i] = chars[j];
    chars[j] = temp;
}
}

```

105) Given a 2D grid of characters, you have to search for all the words in a dictionary by moving only along two directions, either right or down. Print the word if it occurs.

Sample :

```

a z o l
n x h o
v y i v
o r s e

```

Dictionary = {van, zoho, love, are, is}

Output:

```

zoho
love
Is

```

ANSWER;

```

import java.util.HashSet;
import java.util.Set;

public class WordSearchInGrid {
    public static void main(String[] args) {
        char[][] grid = {
            {'a', 'z', 'o', 'l'},
            {'n', 'x', 'h', 'o'},
            {'v', 'y', 'i', 'v'},
            {'o', 'r', 's', 'e'}
        }
    }
}

```

```

};
String[] dictionary = {"van", "zoho", "love", "are", "is"};

System.out.println("Output:");
searchWordsInGrid(grid, dictionary);
}

public static void searchWordsInGrid(char[][] grid, String[] dictionary) {
    Set<String> wordsFound = new HashSet<>();
    for (String word : dictionary) {
        if (searchWord(grid, word)) {
            wordsFound.add(word);
            System.out.println(word);
        }
    }
}

public static boolean searchWord(char[][] grid, String word) {
    int rows = grid.length;
    int cols = grid[0].length;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (searchFromCell(grid, word, i, j, 0)) {
                return true;
            }
        }
    }
    return false;
}

private static boolean searchFromCell(char[][] grid, String word, int row, int col, int index) {
    if (index == word.length()) {
        return true;
    }

    int rows = grid.length;
    int cols = grid[0].length;

    if (row >= rows || col >= cols) {
        return false;
    }

    if (grid[row][col] == word.charAt(index)) {

```

```

        // Move right
        if (searchFromCell(grid, word, row, col + 1, index + 1)) {
            return true;
        }
        // Move down
        if (searchFromCell(grid, word, row + 1, col, index + 1)) {
            return true;
        }
    }

    return false;
}
}

```

106) Given a string, change the order of words in the string (last string should come first).

Should use RECURSION

Sample: one two three

ANSWER;

```

public class ReverseWordsRecursively {
    public static void main(String[] args) {
        String input = "one two three";
        String reversed = reverseWordsRecursively(input);
        System.out.println("Output: " + reversed);
    }

    public static String reverseWordsRecursively(String input) {
        if (input.isEmpty()) {
            return input;
        }

        // Split the input string into words
        String[] words = input.split(" ");

        // Reverse the order of words except the last one recursively
        String reversed = reverseWordsRecursivelyHelper(words, words.length - 2);

        // Append the last word
        reversed += " " + words[words.length - 1];

        return reversed;
    }
}

```

```
private static String reverseWordsRecursivelyHelper(String[] words, int index) {  
    if (index < 0) {  
        return "";  
    }  
  
    // Recursively reverse the order of words  
    String reversed = reverseWordsRecursivelyHelper(words, index - 1);  
  
    // Append the current word  
    reversed += " " + words[index];  
  
    return reversed;  
}  
}
```

* **END** *